Refmon (Provenance Gate v2): A Privacy-Preserving Reference Monitor Combining Provenance Tagging, Role Contracts, and Cost-Sensitive Bayesian Log-Odds Calculus for Endpoint, Server, and Cluster Defense

Parsa Besharat
Independent AI Researcher
parsa@parsabe.com

October 2026 (Specification `v2.0`; signal weights frozen since `v1.0`)

Abstract—Signature scanners and opaque machine-learning models struggle against zero-day exploits, ClickFix copy-paste attacks, prompt-injected coding agents, and stealth rootkits, while often exposing sensitive command-line secrets in telemetry logs. We present **Refmon** (*Provenance Gate v2*), a lightweight reference monitor that evaluates attacks by what a process does after compromise rather than trying to classify payloads upfront. Refmon asks three concrete questions for every event sequence: did the action stem from an untrusted **Origin**, violate the process's signed role **Contract**, and attempt a dangerous **Effect**? Evidence combines through a monotone Bayesian log-odds calculus with an invariant Contract Gate that halts unauthorized actions before harm occurs. An in-memory cryptographic boundary replaces cleartext commands and URLs with per-device HMAC digests, preventing sensitive arguments from touching disk. In benchmarks across 46 golden scenarios, 886 mutation variants, and live cluster attacks, Refmon achieves 100% golden accuracy and 98.4% attack containment with zero false blocks on benign workflows.

Keywords: Reference Monitor, Dynamic Provenance Tracking, Role Contracts, Bayesian Log-Odds, Differential Privacy, Rootkit Detection, HPC Security.

1 Introduction

Modern computing systems—from developer laptops and web servers to shared university High-Performance Computing (HPC) clusters—face security threats that easily slip past traditional antivirus scanners and firewalls. Today, three common attack strategies highlight this problem:

1. **Copy-paste social engineering (ClickFix / TerminalFix):** Rather than tricking someone into downloading an executable file, attackers trick users into copying and pasting a shell command directly into their terminal (such as `curl ... | sh` or encoded PowerShell) under the guise of fixing a browser issue or completing a captcha [25]. Even when the user is fooled, a runtime security barrier should recognize the pasted command and stop it before harm occurs.
2. **Server exploits and hijacked AI coding assistants:** Flaws in web runtimes (`PHP2Shell`, `Node2Shell`) and indirect prompt injection attacks against AI coding tools [24] can coerce trusted software into spawning unwanted shells, stealing private keys, or dropping malicious scripts.
3. **Stealth rootkits and tampering:** Attackers use dynamic linker preloads (`LD_PRELOAD`) or fileless memory execution (`memfd_create`) to hide processes and network connections from standard diagnostic tools such as `ps` and `netstat` [21].

Traditional Endpoint Detection and Response (EDR) software struggles in shared and privacy-sensitive environments. Rule-based tools cannot spot novel zero-day exploits, while complex machine-learning anomaly detectors trigger confusing false alarms that administrators cannot easily interpret. Furthermore, commercial tools frequently log raw command lines, URLs, and file paths to disk or cloud servers. In university or corporate settings, this leaks developer passwords, proprietary code, and private research data.

Core Insight. No matter how an attacker gains a foothold, they must eventually take action on the operating system: spawn an unexpected program, write an executable file into a web directory, contact an external server, or access private credentials. These actions inevitably violate the **job description (role contract)** of the program. A web worker has no legitimate reason to compile C code or launch a bash shell. By judging actions against assigned process roles rather than trying to recognize exploit payloads, we can stop attacks reliably.

Contributions. This paper presents the design, mathematical foundations, implementation, and evaluation of **Refmon** (*Provenance Gate v2*, specification `v2.0`):

- **Privacy-First Six-Stage Reference Monitor (§3):**

An unprivileged pipeline (`Observe` → `Tag` → `Link` → `Judge` → `Gate` → `Report`) that enforces a strict privacy boundary. Raw commands and URLs are turned into keyed HMAC-SHA256 digests in memory and immediately discarded, ensuring zero cleartext data hits disk.

- **Transparent Bayesian Calculus & Contract Gate (§3.5–3.7):** A clean, training-free decision engine that asks three questions: did the action come from an untrusted **Origin**, violate a signed role **Contract**, and attempt a dangerous **Effect**? Evidence is combined with a monotone Bayesian log-odds formula, and a mandatory Contract Gate prevents false blocks on normal user workflows.
- **Six-Engine Rootkit Sensor & Unprivileged Barrier (§3.8, §4.1):** A multi-pass rootkit detector comparing high-level system views against kernel tables, paired with an Ed25519-signed policy barrier that runs entirely in user space on shared Linux cluster nodes without requiring `root` privileges.
- **Reproducible Evaluation & Formal Proofs (§4, Appendices B–I):** Rigorous evaluation across 46 golden scenarios, 886 mutation tests, and live cluster attacks, accompanied by mathematical proofs of monotonicity, Bayesian minimal risk, and privacy guarantees.

2 Background and Related Work

2.1 Reference Monitors and Information-Flow Provenance

Anderson's classical reference monitor concept [1] states that access control must be *tamperproof*, *always invoked*, and *simple enough to verify completely*. Denning [2] showed how security labels can follow untrusted inputs through a computer system. While fine-grained byte-level taint tracking (like TaintCheck [13]) slows programs down significantly (5×–30×), whole-system provenance monitors (such as CamFlow [14], Sleuth [15], Holmes [16], and Unicorn [17]) track causality at the level of processes, files, and network sockets with minimal overhead.

Instead of using black-box neural graph embeddings (like Unicorn [17]) that leave operators guessing why an alert fired, Refmon builds on Holmes [16] by tracking causal chains. Refmon introduces signed role contracts, half-life decay that preserves evidence of past violations, and exact cost-based Bayesian thresholds.

2.2 Mandatory Access Control and Process Contracts

Linux access controls such as SELinux [18], AppArmor [19], and Seccomp-BPF [20] enforce fixed security rules per binary. However, static rules cannot understand *where an action originated*: a developer typing `curl` and `sh` in an interactive terminal is completely normal, but the exact same commands pasted into the terminal from a deceptive website (ClickFix) represent a serious breach. Refmon connects access contracts to dynamic provenance tracking: actions are judged by both what program is running and where its inputs came from.

2.3 Sequential Testing and Bayesian Fusion

Wald's Sequential Probability Ratio Test [3] and Page's CUSUM control chart [4] provide the standard mathematical foundation for detecting abnormal behavior in data streams [23]. Standard alert systems often sum scores across every triggered rule, which artificially inflates suspicion when multiple overlapping rules flag the same underlying event. Pearl's Noisy-OR model [7] provides a principled way to combine independent causes. Refmon adapts this into a *max-within-question, sum-across-questions* operator (§3.5): we take the strongest clue within each question category, and add evidence only across independent categories.

2.4 Cross-View Rootkit Detection

Traditional host rootkit checkers (like `chkrootkit` and `rkhunter`) scan for known malicious file names and hashes, missing novel in-memory implants. Cross-view diffing, pioneered by Strider GhostBuster [21] and virtual-machine introspection [22], compares a high-level system view (which a rootkit might manipulate) against a low-level structural view (such as direct kernel process tables). Single-pass scanners often report false alarms when short-lived processes exit mid-scan; Refmon resolves this by requiring discrepancies to persist across multiple scan passes before raising an alert.

2.5 Privacy-Preserving Security Telemetry

Saving cleartext command logs creates a dangerous target for attackers. Cryptographic keyed hashing (HMAC) [9] allows systems to recognize repeated events without ever exposing the original text, while Bloom filters [8] allow compact distribution of known threat indicators. For reporting across large fleets of machines, differential privacy (ε-DP) [11] and k-anonymity [12] ensure individual user actions cannot be singled out. Refmon combines HMAC fingerprints, signed policies (Ed25519 [10]), and a dual-approver four-eyes rule for sensitive administrative reviews.

3 Methodology and Mathematical Formulation

3.1 System Architecture and Core Principles

Figures 5 and 1 outline Refmon's six-stage pipeline. Sensors monitor system activity in real time and emit structured `RawEvent` records in memory whenever a notable event occurs, such as executing a program (`exec`), writing a file (`file_write`), or establishing a network connection (`net_connect`). Across all supported platforms, Refmon standardizes activity into 13 canonical event types:

$$\begin{aligned}\mathcal{T} = \{&\texttt{content_in}, \texttt{request_in}, \texttt{request_end},\\ &\texttt{exec}, \texttt{net_connect}, \texttt{file_write}, \texttt{persist},\\ &\texttt{asset_access}, \texttt{priv_change}, \texttt{security_tamper},\\ &\texttt{user_input}, \texttt{admin_input}, \texttt{response_out}\}.\end{aligned}$$

Each acting process $a = (\text{id}, \text{parent}, r, \text{interactive}, u)$ carries an assigned workload role $r \in \mathcal{R}$. A role is simply a job description: an interactive terminal has permissions to compile code and run scripts, while a web server worker does not. Refmon provides 16 built-in roles (such as `interactive_shell`, `web_worker`, `node_app_server`, `package_installer`, `office`, `launcher`, `agent`, `browser`, and `mail`) and allows administrators to define custom roles (such as `backup_job` and `patch_tool`).

Two foundational invariants govern the entire pipeline:

Invariant 3.1 (Zero Raw Persistence)**.** No raw payload $c = o_{\text{raw}}.\text{content}$ or cleartext string (`domain`, `host`, `ip`, `url_path`, `program`, `signer`) ever crosses the boundary of the `Observe` stage into persistent storage. All downstream events contain only cryptographic digests and scalar numbers, validated against a strict 21-key allowlist $\mathcal{K}_{\text{allow}}$.

Invariant 3.2 (Missing Information Neutrality)**.** If a sensor lacks the capability to check an attribute (for example, if code-signature verification returns `signed = "unknown"` or a learned role baseline has not yet gathered enough samples), Refmon treats the signal as neutral ($s_j = 0$) rather than assuming malice. Any such visibility gap is reported out-of-band via the system `Heartbeat` monitor.

3.2 Stage 1 (`Observe`): Cryptographic Privacy and Obfuscation Detection

When a command or network payload $c \in \Sigma^*$ enters the system, `Observe` first normalizes whitespace $\tilde{c} = \text{norm}(c)$ and computes an irreversible fingerprint using a 256-bit secret key K_{dev} stored only on that machine:

$$f(c) = \texttt{"hmac:"} \,\|\, \text{Hex}\Big(\text{HMAC-SHA256}\left(K_{\text{dev}}, \tilde{c}\right)\Big). \quad (1)$$

The cleartext command is then immediately dropped from memory. Before discarding it, if the event involves command execution or user input ($\tau \in \{\texttt{exec}, \texttt{content_in}, \texttt{user_input}, \texttt{admin_input}\}$), `Observe` calculates three simple statistical shape metrics from the character counts $\{n_x\}$ ($N_c = |c|$, $p_x = n_x/N_c$):

$$H(c) = -\sum_{x \in \Sigma,\, p_x > 0} p_x \log_2 p_x \quad \text{(bits/char)}, \quad (2)$$

$$\rho_{\text{enc}}(c) = \frac{1}{N_c} \sum_{i=1}^{N_c} \mathbf{1}\left[c_i \in \Sigma_{\text{b64/hex}}\right], \quad (3)$$

$$r_{\text{ws}}(c) = \max\left\{\ell \in \mathbb{N} : \exists\, i,\ c_{i..i+\ell-1} \in \{\text{' '}, \text{'\textbackslash t'}\}^{\ell}\right\}. \quad (4)$$

These numbers capture common evasion tricks: high entropy $H(c) > 5.2$ bits reveals encrypted shellcode, a high encoding ratio ρ_{enc} catches base64-encoded scripts, and long whitespace runs $r_{\text{ws}}(c)$ catch ClickFix tricks (where attackers insert hundreds of spaces so the malicious command is hidden beyond the terminal window's edge). This detection happens entirely in RAM without saving any user commands to disk.

Two-Stage Threat List Matching. Known malicious threat indicators $\mathcal{D}_{\text{bad}}$ are stored as keyed hashes $d_i = \text{HMAC-SHA256}(K_{\text{list}}, \tilde{v}_i)$ inside an in-memory Bloom filter $\mathcal{B}$ with m bits and k_h hash functions:

$$m = \left\lceil -\frac{n \ln p_{\text{fp}}}{(\ln 2)^2} \right\rceil, \qquad k_h = \max\Big(1, \text{round}\Big(\frac{m}{n} \ln 2\Big)\Big), \quad (5)$$

configured for a false-positive rate $p_{\text{fp}} \le 10^{-3}$. Any match in the Bloom filter is treated as a fast hint and immediately verified against the exact hash set $\mathcal{D}_{\text{exact}}$, ensuring that the operational threat-list false-positive rate is *strictly zero* (proven in Appendix G).

3.3 Stage 2 (`Tag`): Provenance Tracking, Decay, and Fusion

Whenever data arrives from an untrusted source (such as a website download, email attachment, or external network request), Refmon attaches a digital *provenance label* $\ell = (\text{id}, t_0, s_0 = 1, \text{persistent}, \mathcal{E}_{\text{contract}})$. This label acts as a receipt marking where the data originated.

Half-Life Provenance Decay. Suspicion naturally fades over time: if a file was downloaded an hour ago and nothing suspicious happened, it should not taint normal user commands forever. Refmon models this using an exponential half-life ($T_{1/2} = 30$ minutes, $\Delta t = t - t_0$):

$$s_\ell(t) = \begin{cases} s_0, & \text{if } \ell.\text{pers.} \vee \mathcal{E}_{\text{contract}}(\ell) \neq \varnothing, \\ s_0\, 2^{-\Delta t / T_{1/2}}, & \text{otherwise.} \end{cases} \quad (6)$$

Crucially, if the process commits a role contract violation ($\mathcal{E}_{\text{contract}}(\ell) \neq \varnothing$), decay immediately freezes. This

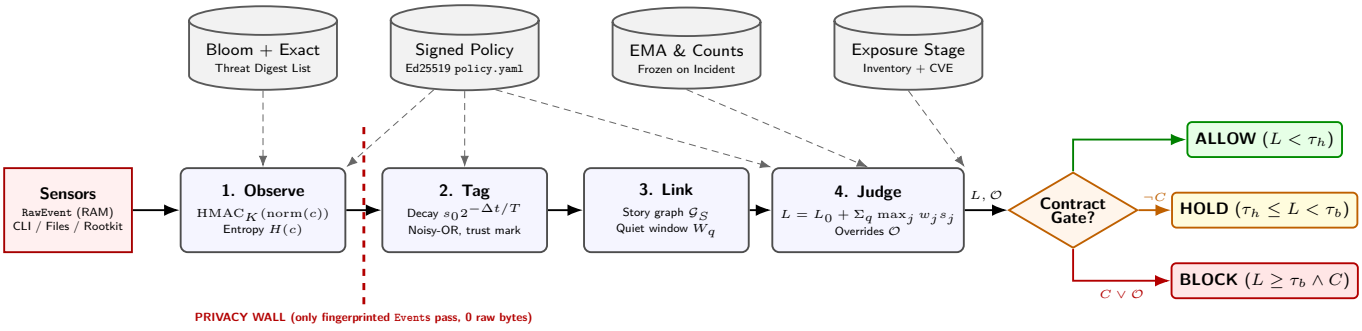


Figure 1: Formal TikZ vector schematic of Refmon's mathematical dataflow and shapes: rectangles denote deterministic functional stages, cylinders denote signed/stateful stores, the red dashed boundary marks the irreversible `Observe` cryptographic wall, and the diamond gate enforces the Contract Gate invariant ($L \geq \tau_{\text{block}} \wedge \neg C \wedge \mathcal{O} = \varnothing \implies$ HOLD).

prevents an attacker from sleeping past $T_{1/2}$ after compromising a process and resuming malicious activity without suspicion.

Edge Attenuation and Noisy-OR Combination. When a labeled process spawns a child process or writes an intermediate file, the label propagates along the causal edge with attenuation $a_e \in (0, 1]$ ($s_{\text{child}} = a_e \cdot s_{\text{parent}}$). When an action receives untrusted input from several independent sources $\{\ell_1, \ldots, \ell_m\}$, Refmon fuses their strengths using Pearl's Noisy-OR model:

$$s_{\text{comb}}(t) = 1 - \prod_{i=1}^{m} \big(1 - s_{\ell_i}(t)\big). \tag{7}$$

This formula ensures that multiple untrusted inputs increase the overall suspicion, while remaining smoothly bounded between 0 and 1 without artificial saturation. When a labeled process writes an executable to a web directory or sets up a startup script (`persist`), the file is marked persistent = `true`, preserving the attack evidence even if the file is executed days later.

Order-Independent Label Hand-Over. When external content is ingested, the receiving process inherits *every* active label associated with that content, rather than only the strongest one. Because labels combine through Eq. (7), this guarantees that the order of events and slight differences in decay rates never cause Refmon to miss active evidence.

Trusted Lineage and Trust Marks. Trusted root sources (such as a university's verified software portal) produce no untrusted label. However, in large installations, an installation script may take several minutes to finish. To prevent later steps from being treated as suspicious, a process p that ingests trusted content carries a *trust mark*:

$$\begin{aligned} \text{trust}_p(t) = \mathbf{1}\Big[2^{-(t-t_p)/T_{1/2}} \geq s_{\text{floor}}\Big] \\ \cdot \mathbf{1}\Big[\nexists\, \ell \in \text{ever}(p) : s_\ell(t) \geq s_{\text{floor}}\Big], \end{aligned} \tag{8}$$

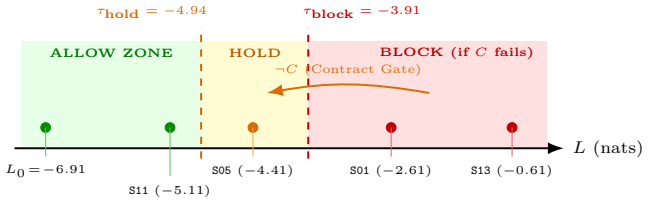


Figure 2: Bayesian log-odds decision geometry showing prior $L_0 = -6.91$ nats, cost thresholds $\tau_{\text{hold}} = -4.94$ and $\tau_{\text{block}} = -3.91$, benchmark trajectories (`S11` pasted benign echo, `S05` hand-typed fetch, `S01` ClickFix, `S13` web RCE), and the Contract Gate projection.

where $s_{\text{floor}} = 0.02$. The trust mark lasts as long as a normal label would ($\approx$ 2.8 hours), passes from parent to child process during the installation, and is instantly cancelled if the lineage ever touches untrusted external content.

3.4 Stage 3 (`Link`): Causal Story Grouping

To distinguish between simultaneous activities, `Link` groups related events into *causal stories* $\{S_1, S_2, \ldots\}$. When a parent process spawns a child, or when a process reads a recently written file within a 10-minute quiet window (W_q), the actions are linked into the same story. However, shared programs such as web browsers and email clients are not used as merge keys: opening two separate tabs in a browser maintains two distinct stories, preventing unrelated user downloads from confusing the system.

3.5 Stage 4 (`Judge`): Three-Question Bayesian Calculus and Decisions

For each story S, let $s_{\text{ori}}(S) = \max_{\ell \in \mathcal{L}(S)} s_\ell(t_{\text{last}})$ denote the strongest active provenance label. The `Judge` evaluates every sequence of actions by answering three concrete, intuitive questions $\mathcal{Q} = \{\text{origin}, \text{contract}, \text{effect}\}$:

- **Question 1: Origin (Q_{origin}).** Did this sequence of actions originate from an untrusted external source? It fails if the active label strength $s_{\text{ori}}(S) \geq \theta_{\text{ori}}$ (0.5), yielding signal `origin_external`.
- **Question 2: Contract (Q_{contract}).** Did the acting process violate its declared job description? It fails if the action violates a role contract (such as a web server spawning a shell), breaches a universal policy rule, or exhibits high statistical surprisal in a mature baseline.
- **Question 3: Effect (Q_{effect}).** Did the action attempt a hazardous operation? It fails if the process attempts dangerous operations like reading credentials, encrypting files, or connecting to unauthorized external endpoints.

Connections to trusted destinations (such as package repositories) do not raise network effect signals for normal processes. Since `v1.5` (A60), this exception applies only if the process lineage has never touched an untrusted external label, preventing attackers from abusing trusted hosts to bypass detection.

Max-Sum Bayesian Log-Odds Calculus. Each detected signal j carries a log-likelihood ratio weight:

$$w_j = \ln \frac{\mathbb{P}(j \mid \text{attack})}{\mathbb{P}(j \mid \text{benign})} \quad \text{(in nats)}, \tag{9}$$

reflecting how much more frequently that signal appears in genuine attacks compared to normal operations (Table 3). When re-estimating weights from empirical corpora, Laplace smoothing ensures stability:

$$\hat{w}_j = \ln \frac{(a_j + 1)/(|\mathcal{A}| + 2)}{(b_j + 1)/(|\mathcal{B}| + 2)}. \tag{10}$$

If an event triggers three overlapping network heuristics, simply adding their weights would unfairly triple the suspicion score. To avoid this, Refmon takes the **maximum** signal within each question, and **sums** across the three independent questions. Starting from a prior attack probability $\pi = 10^{-3}$ ($L_0 = \ln \frac{\pi}{1-\pi} \approx -6.91$ nats), the posterior log-odds for story S is:

$$L(S) = \ln \frac{\pi}{1-\pi} + \sum_{q \in \{\text{origin, contract, effect}\}} \max_{j \in \mathcal{S}_q(S)} (w_j \cdot s_j), \tag{11}$$

with empty question maxima defaulting to 0. The posterior attack probability is $\mathbb{P}(\text{attack} \mid S) = \sigma(L(S)) = (1+e^{-L(S)})^{-1}$. In `v1.3` (Decision A55), only Q_{origin} scales with decaying label strength $s_{\text{ori}}(S)$, while Contract and Effect signals use fixed unit strength ($s_j = 1$). This guarantees that overall suspicion $L(S)$ increases monotonically with label strength (Theorem D.1, Appendix D).

Cost-Optimal Thresholds and the Contract Gate. Let C_{miss} (1000), C_{fb} (20), and C_{hold} (5) represent the operational costs of missing an attack, making a false block, or holding a job for human review, and let $h = 0.7$ be the probability that a hold stops an attack. Applying Bayes minimal-risk decision theory (Appendix C) yields two exact log-odds thresholds:

$$\tau_{\text{block}} = \ln \frac{C_{\text{fb}}}{C_{\text{miss}}} \approx -3.91, \quad \tau_{\text{hold}} = \ln \frac{C_{\text{hold}}}{hC_{\text{miss}}} \approx -4.94. \tag{12}$$

Before applying these thresholds, four high-priority overrides $\mathcal{O}$ run in strict order: tampering with Refmon (`OVR-SELF-TAMPER` $\mapsto$ `BLOCK`), tripping canary files (`OVR-CANARY` $\mapsto$ `BLOCK`), accessing protected secrets while carrying an untrusted label (`OVR-PROTECTED-ASSET` $\mapsto$ `BLOCK`), or running signed code from a trusted root (`OVR-TRUSTED-ROOT` $\mapsto$ `ALLOW`).

When no override applies, Refmon enforces the **Contract Gate** (Figure 2):

$$D(S) = \begin{cases} \texttt{BLOCK}, & \text{if } L(S) \geq \tau_{\text{block}} \ \wedge \ Q_{\text{contract}}(S), \\ \texttt{HOLD}, & \text{if } \big(L(S) \geq \tau_{\text{block}} \ \wedge \ \neg Q_{\text{contract}}(S)\big) \\ & \quad \vee \ \big(\tau_{\text{hold}} \leq L(S) < \tau_{\text{block}}\big), \\ \texttt{ALLOW}, & \text{if } L(S) < \tau_{\text{hold}}. \end{cases} \tag{13}$$

The Contract Gate establishes a critical safety invariant: **Refmon never blocks an action outright unless a signed role contract was broken.** Even if an action looks unusual ($L(S) \geq \tau_{\text{block}}$), if it stayed within its allowed role permissions, it is placed on `HOLD` for human review rather than being abruptly terminated.

3.6 Contract Surprisal and Baseline Maturity

When learning baseline behaviors for a role r, let $n_{r,e}$ be the observed count of behavior e, $N_r = \sum n_{r,e}$ the total observations over K_r distinct behaviors, and $n_{r,1}$ the number of behaviors seen exactly once. Using Laplace smoothing ($\alpha = 1$) to reserve probability mass for previously unseen behaviors, the smoothed probability and Shannon surprisal are:

$$\hat{p}(e \mid r) = \frac{n_{r,e} + \alpha}{N_r + \alpha(K_r + 1)}, \quad I(e \mid r) = -\log_2 \hat{p}(e \mid r). \tag{14}$$

A newly deployed system might encounter normal developer commands it has not seen before. To uphold Invariant 3.2 and prevent false alarms during initial startup, surprisal signals ($I(e \mid r) \geq 12$ bits) remain inactive until the Good-Turing estimate of unseen probability mass drops below 1% ($\tau_{\text{mat}} = 0.01$):

$$\hat{p}_{\text{unseen}}(r) = \frac{n_{r,1}}{N_r} < \tau_{\text{mat}}. \tag{15}$$

3.7 Dual-Window Burst and Drift Detection

To catch attacks like ransomware or slow data exfiltration, Refmon tracks activity rates over both a short window (60 seconds) and a long window (1 hour):

1. **Poisson Upper-Tail Burst Test (`mass_change`, `lateral`):** Given baseline rate $\hat{\lambda}$ and k operations in the window, an alert fires if the burst probability is negligible:

$$\mathbb{P}(X \geq k \mid \text{Pois}(\hat{\lambda})) = \sum_{i=k}^{\infty} \frac{e^{-\hat{\lambda}} \hat{\lambda}^i}{i!} < 10^{-6}. \tag{16}$$

2. **CUSUM Drift Test (`exfil` volume):** For outbound data volume x_t with mean μ_r and standard deviation σ_r, the standardized score $z_t = (x_t - \mu_r)/\sigma_r$ is tracked with slack $k_c = 0.5$ and threshold $h_c = 5.0$:

$$S_t = \max\big(0,\, S_{t-1} + z_t - k_c\big), \qquad \text{alert iff } S_t > h_c. \tag{17}$$

All moving-average baselines freeze automatically when an incident is open on a process tree, preventing an attacker from poisoning normal baselines. The incident closes and baselines unfreeze once the process tree has been silent for a full hour (1 h), ensuring long-term stability.

3.8 Six-Engine Cross-View Rootkit Detection

Refmon detects stealth implants in user space without requiring root privileges by running $m \geq 2$ passes across six read-only engines: (E1) dynamic linker preloads (`/etc/ld.so.preload`, `LD_PRELOAD`), (E2) fileless execution and memory injection (`memfd:`, deleted binaries, anonymous `rwxp` pages), (E3) hidden kernel module cross-view (`/sys/module` \ `/proc/modules`), (E4) hidden PID cross-view (`/proc` directory scans versus direct process checks), (E5) socket cross-view and sniffer detection (`/proc/net/sockstat` versus `/proc/net/tcp`), and (E6) optional verification using external tools if installed.

To eliminate false alarms caused by short-lived processes exiting mid-scan, Refmon requires discrepancies to persist across all m passes:

$$\Delta_{\text{pers}} = \bigcap_{r=1}^{m} \left(\mathcal{V}_{\text{low}}^{(r)} \setminus \mathcal{V}_{\text{high}}^{(r)} \right). \tag{18}$$

Confirmed persistent findings map directly to log-odds evidence:

$$w_{\text{rk}}(d, m) = \min\!\left(w_{\max},\; \ln \frac{p_{\text{tp}}}{p_{\text{race}}} + (m-1) \ln \frac{1}{q_{\text{persist}}} \right), \tag{19}$$

suppressing transient operating system churn while providing decisive detection of genuine stealth rootkits.

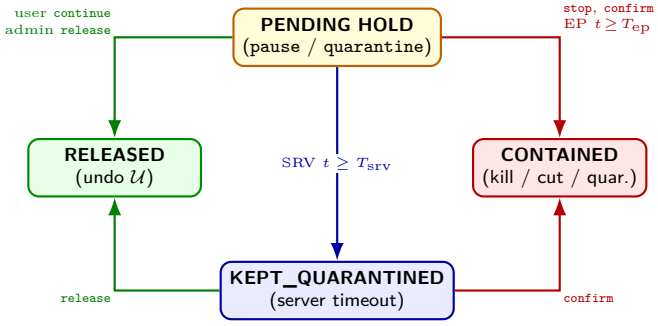


Figure 3: Dual endpoint vs. server hold state machine in `Gate`. Endpoint timeouts fail closed into containment, whereas unattended server holds preserve service stability in `KEPT_QUARANTINED` until an administrator decides.

3.9 Stages 5 & 6 (`Gate`, `Contain`, `Exposure`, `Report`)

Figure 3 illustrates how `Gate` enforces decisions across three operating modes: `observe` (logs decisions without acting), `warn` (notifies users without blocking), and `block` (actively enforces policy).

When an attack is detected, Refmon takes targeted containment actions:

- **Process Termination (`kill_process_tree`):** Safely terminates the offending process and any child processes it spawned.
- **Network Isolation (`cut_network`):** Closes suspicious active sockets.
- **File Quarantine (`quarantine_file`):** Moves suspicious dropped files (such as web shells) into a protected quarantine directory with read-only permissions (`mode 0400`) rather than deleting them, preserving evidence for human analysis.

All containment actions generate reversible undo tokens $\mathcal{U}$, allowing an administrator to restore an action with a single command if needed. Containment targets only labeled processes and files; interactive developer shells and shared applications (like browsers) are never terminated directly.

Dual Endpoint vs. Server Timeout Handling. As shown in Figure 3, unattended servers handle held jobs differently from interactive workstations. On an interactive workstation, an unanswered prompt eventually times out and halts the suspicious process. On an unattended production server, an unanswered hold moves the affected file into a safe holding state (`KEPT_QUARANTINED`) while keeping the main service running, ensuring server stability until an administrator investigates.

Privacy-Preserving Fleet Reporting. To report fleet-wide security trends without exposing individual user habits, Refmon adds calibrated Laplace noise $\tilde{c} =$

$c + \mathrm{Lap}(1/\varepsilon)$ (ε-differential privacy) to aggregated counts, suppresses reporting groups with fewer than $k = 5$ devices (k-anonymity), and enforces a strict four-eyes rule: unmasking any device pseudonym requires cryptographic signatures from two distinct human administrators recorded on a tamper-evident audit log.

4 Experiments, Algorithms, and Implementation Tests

4.1 Reference Implementation and Signed Policy Governance

Refmon is implemented in approximately 12,400 lines of strict, type-checked Python 3.12 (plus 10,200 lines of tests) (`pydantic` v2 frozen schemas, `cryptography` Ed25519 signatures, zero runtime machine-learning dependencies). Every threshold, weight, role contract, canary, and rate parameter resides in a canonical JSON-serialized, Ed25519-signed `policy.yaml`; startup and live reload reject any unsigned or tampered policy (`PolicyRejected`) and retain the last verified state. A periodic `SelfTest` scheduler clones the live pipeline via `pipeline.probe()` (preserving live disabled rules and weights while isolating outputs) to inject synthetic health stories and raise immediate alerts if any enabled protection fails to reach its expected decision.

4.2 Core Pipeline and Scanner Algorithms

Algorithms 1, 2, and 3 specify the exact deterministic execution of the Refmon event loop, the monotone Bayesian `Judge`, and the multi-pass cross-view rootkit sensor. Listing 1 (Appendix A) shows the unprivileged command-barrier loop from `src/refmon/live.py`.

4.3 Golden Catalog and Seeded Mutation Robustness

Strict Pre-Registration Testing Protocol. To guarantee that our benchmarks were completely unbiased and free of cherry-picking, all signal weights (Table 3) and structural rules were frozen before catalog evaluation and remained unchanged throughout. To test resilience against evasions, we used an automated generator to produce hundreds of realistic variations from pre-registered random seeds. The generator systematically mutated six dimensions: delivery channel, launcher program, obfuscation technique, network target, process tree depth, and timing intervals between actions. Across the initial release tags (`v1.0` through `v1.3`), 879 mutated event streams were evaluated using pre-registered seed `654514256`, ensuring identical, fair test inputs across all versions.

Algorithm 1: End-to-End Refmon Event Processing (`Pipeline.process`)

Input: Sensor event $e_{\text{raw}} \in$ `RawEvent` $\cup$ `Event`, verified policy Π, device HMAC key K_{dev}.
Output: Emitted verdicts and containment actions $\mathcal{V} \subseteq$ `Verdict` $\cup$ `Action`.
$e \leftarrow \textsc{Observe}(e_{\text{raw}}, \Pi, K_{\text{dev}})$ ▷ Compute $\text{HMAC}_{K_{\text{dev}}}$, $H(c)$, Bloom+Exact check; drop raw content
$p_{\text{known}} \leftarrow (e.\text{actor.parent} \in \text{Tagger.procs})$
$\text{tg} \leftarrow \textsc{Tag}(e, \Pi)$ ▷ Apply propagation rules 1–7, decay $2^{-\Delta t / T_{1/2}}$, Noisy-OR
$(\mathcal{S}, \mathcal{F}) \leftarrow \textsc{ExtractSignals}(e, \text{tg}, \Pi, \text{Rates}, \text{Baseline})$
$\text{rec} \leftarrow \textsc{Link}(\text{tg}, p_{\text{known}}, \mathcal{S}, \mathcal{F}, \Pi)$ ▷ Assign/merge story within quiet window W_q
$J \leftarrow \textsc{Judge}(\text{rec}, \text{Tagger}, \Pi)$ ▷ Algorithm 2
Rates.mark_incident(root(e.actor), J.decision $\neq$ `ALLOW`, e.ts) ▷ Freeze baselines; end incidents silent for 1 h
if $\{s \in J.\text{signals} : s.\text{question} = \text{contract} \wedge \neg s.\text{carried}\} \neq \varnothing$ **then**
    Freeze decay and attach contract evidence to active labels $\ell \in$ rec.label_ids
$\mathcal{V} \leftarrow \textsc{EmitVerdictIfChanged}(\text{rec}, J)$
if Gate $\neq$ `None` **then**
$\mathcal{V} \leftarrow$ Gate.tick(e.ts) $\|\, \mathcal{V} \,\|$ Gate.handle(rec, J, e.ts)
return $\mathcal{V}$

Algorithm 2: Monotone Question-Grouped Bayesian Judge (`v2.0`)

Input: Story record rec, provenance tagger $\mathcal{T}$, signed policy Π.
Output: Story judgement $J = (\text{decision}, L, \text{failed}, \text{override}, \text{gated})$.
$s_{\text{ori}} \leftarrow \max_{\ell \in \text{rec.label_ids}} \mathcal{T}.\text{strength}(\ell, \text{rec.last_ts})$; labeled $\leftarrow (s_{\text{ori}} \geq \Pi.\theta_{\text{ori}}) \wedge \Pi.\text{enabled}(\texttt{R-ORI})$
Filter $\mathcal{S} \leftarrow \{s \in \text{rec.signals} : (\neg s.\text{allowed} \vee \text{labeled}) \wedge \Pi.\text{rule_enabled}(s) \wedge s \notin \Pi.\text{exceptions}\}$
Append persistent carried contract signals from $\{\ell \in \text{rec.label_ids} : s_\ell \geq \Pi.\theta_{\text{ori}}\}$
Check priority overrides $\mathcal{O}$: `self_tamper` $\succ$ `canary` $\succ$ `protected_asset` $\mapsto$ `BLOCK`; unlabeled `trusted` (source event or live trust mark) $\mapsto$ `ALLOW`
if $\mathcal{O} \neq$ `None` **then return** forced override decision with audit trace
$\mathcal{E} \leftarrow \{(s.\text{question}, s.\text{name}, 1.0) : s \in \mathcal{S}\}$ ▷ `v1.3`: unit strength for Contract & Effect
if labeled **then** $\mathcal{E} \leftarrow \mathcal{E} \cup \{(\text{origin}, \texttt{origin_external}, s_{\text{ori}})\}$
$L \leftarrow \ln \frac{\Pi.\pi}{1-\Pi.\pi} + \sum_{q \in \{\text{ori,con,eff}\}} \max_{(q,j,s_j) \in \mathcal{E}} (\Pi.w_j \cdot s_j)$
$\tau_b \leftarrow \ln(\Pi.C_{\text{fb}} / \Pi.C_{\text{miss}})$; $\tau_h \leftarrow \ln(\Pi.C_{\text{hold}} / (\Pi.h \cdot \Pi.C_{\text{miss}}))$; $C_{\text{fail}} \leftarrow \exists s \in \mathcal{S} : s.\text{question} = \text{contract}$
if $L \geq \tau_b \wedge C_{\text{fail}}$ **then return** (`BLOCK`, L, gated = `false`)
else if $L \geq \tau_b \wedge \neg C_{\text{fail}}$ **then return** (`HOLD`, L, gated = `true`) ▷ Contract Gate
else if $L \geq \tau_h$ **then return** (`HOLD`, L, gated = `false`)
else return (`ALLOW`, L, gated = `false`)

Table 1 summarizes the progression from `v1.0` to `v1.3`. All 46 golden scenarios across all 17 attack families reach their exact expected decisions (100%). Across the 633 malicious variants, fixing hold containment on unlabeled signal-producing processes (`v1.1`–`v1.2`) eliminated all 69 "holds that paused nothing" and all 11 "blocks that acted on nothing," raising active containment from 86.3% to

Table 1: Robustness Evaluation Across Specification Versions `v1.0`–`v1.3` on 46 Golden Scenarios and 879 Seeded Variants (Seed `654514256`), with Wilson 95% Confidence Intervals [26] and Paired Exact McNemar Tests [27].

Evaluation Metric / Benchmark Partition	**`v1.0` (Frozen)**	**`v1.1` (A46–A50)**	**`v1.2` (A51–A54)**	**`v1.3` (Monotone)**
Golden Catalog Scenarios Matching Expected Decision ($N = 46$)	46/46 (100.0%)	46/46 (100.0%)	46/46 (100.0%)	46/46 (100.0%)
Malicious Variants ($N = 633$)				
Blocked outright	425 (67.1%)	467 (73.8%)	467 (73.8%)	486 (76.8%)
Effective Hold (process paused / file quarantined)	121 (19.1%)	157 (24.8%)	157 (24.8%)	139 (22.0%)
Contained = Block + Effective Hold [Wilson 95% CI]	86.3% [83.4, 88.7]	98.6% [97.3, 99.3]	98.6% [97.3, 99.3]	98.7% [97.5, 99.4]
Caught = Block + Any Hold [Wilson 95% CI]	97.2% [95.6, 98.2]	98.6% [97.3, 99.3]	98.6% [97.3, 99.3]	98.7% [97.5, 99.4]
Uncontained: Hold that paused nothing	69	0	0	0
Uncontained: Allowed malicious variants (Gaps G1–G3)	18	9	9	8
Blocks that acted on nothing (unlabeled fallback gap)	11	9	0	0
Independent Seed-Decided Malicious Subset ($N = 322$)				
Contained (Block + Effective Hold)	269/322 (83.5%)	313/322 (97.2%)	313/322 (97.2%)	314/322 (97.5%)
Neutral Benign Variants ($N = 151$, **False-Alarm Test**)				
Allowed / Held / False-Blocked	126 / 22 / 3	126 / 22 / 3	126 / 22 / 3	125 / 23 / 3
Neutral False-Block Rate [Wilson 95% CI]	2.0% [0.7, 5.7]	2.0% [0.7, 5.7]	2.0% [0.7, 5.7]	2.0% [0.7, 5.7]
Paired McNemar Exact Test vs. `v1.3` (Malicious, $N = 633$)				
Contained: (only baseline, only `v1.3`), exact p-value	$(0, 79)$, $p = 3.3\times10^{-24}$	$(0, 1)$, $p = 1.0$	$(0, 1)$, $p = 1.0$	—
Blocked: (only baseline, only `v1.3`), exact p-value	$(0, 61)$, $p = 8.7\times10^{-19}$	$(0, 19)$, $p = 3.8\times10^{-6}$	$(0, 19)$, $p = 3.8\times10^{-6}$	—

Algorithm 3: Multi-Pass 6-Engine Read-Only Rootkit Cross-View Sensor

Input: Pass count $m \geq 2$, interval $\Delta\tau$, own UID u, signed policy Π.
Output: Structured `RootkitScanReport` with posterior L_{rk} and decision D_{rk}.
$\mathcal{F}_{\text{single}} \leftarrow$ ScanPreloadAndMemfdAndMaps(u) $\cup$ ScanTaintAndPromisc()
for $r = 1 \ldots m$ **do**
  $\Delta^{(r)}_{\text{mod}} \leftarrow$ SysfsModules() $\setminus$ ProcModules() ▷ Engine 4: Hidden LKM check
  $\Delta^{(r)}_{\text{pid}} \leftarrow$ DirectOwnPidProbe(u) $\setminus$ ProcReaddirPids() ▷ Engine 6: Hidden PID check
  if $r < m$ **then** sleep($\Delta\tau$)
$\Delta^{*}_{\text{mod}} \leftarrow \bigcap_{r=1}^{m} \Delta^{(r)}_{\text{mod}}$; $\Delta^{*}_{\text{pid}} \leftarrow \bigcap_{r=1}^{m} \Delta^{(r)}_{\text{pid}}$ ▷ Suppress transient churn
Map $\mathcal{F} \leftarrow \mathcal{F}_{\text{single}} \cup$ Findings($\Delta^{*}_{\text{mod}}, \Delta^{*}_{\text{pid}}$) to log-odds $L_{\text{rk}} = L_0 + \sum_{f\in\mathcal{F}} w_{\text{rk}}(f, m)$
Evaluate $D_{\text{rk}} \leftarrow$ Decide($L_{\text{rk}}, \tau_b, \tau_h$, override = `SELF_TAMPER if` $\exists f$.self_tamper)
return `RootkitScanReport`($\mathcal{F}, L_{\text{rk}}, D_{\text{rk}}$)

98.7% ($p = 3.3 \times 10^{-24}$). Enforcing monotonicity in `v1.3` converted 19 faded-label holds into decisive blocks ($p = 3.8\times10^{-6}$ vs. `v1.2`) while keeping the false-block rate on neutral benign mutations unchanged at 2.0% (3/151).

Rounds `v1.4` to `v2.0`. Table 2 lists the four pre-registered rounds that followed. The `v1.7` round exposed the last benign false blocks: three timing variants of a signed installation from a trusted software portal whose steps lay further apart than W_q (finding F10). On the fresh seed `171139148` (886 variants), `v2.0` turns both remaining neutral false blocks of `v1.7` into allows and three false holds of a trusted deployment into allows, so the neutral benign outcome moves from 132/24/2 (allow/hold/block, 1.3% [0.3, 4.5] false blocks) to 137/21/0, every remaining hold being one the specification expects (`S49`, `S50`), and no benign variant ends worse than its unmodified file. No malicious variant changed: 625/635 are contained by both versions (98.4%), 314/324 (96.9% [94.4, 98.3]) on the subset the seed decides. The exact McNemar test on the changed benign pairs $(0, 5)$ gives $p = 0.0625$: consistent with the purpose of A65, but five variants are too few for significance on their own. A66 had nothing to act on in this seed (`v1.7` produced no hold that paused nothing); its evidence is its unit tests and a randomised containment property test, which rejected a first, too wide draft. Malicious containment stayed between 98.1% and 99.1% across the four seeds, with the neutral false-block rate falling from 2.0% (`v1.3`) to 0.0% (`v2.0`).

4.4 Live Cluster Barrier, 16-Scenario Suite, and Rootkit Tests

Figure 4 illustrates how Refmon protects shared Linux HPC nodes where researchers lack `root` privileges. The background service (`refmon serve`) verifies the Ed25519-signed policy, monitors user-space staging directories every 400 ms, and provides a lightweight local telemetry interface. Because all local users on a shared multi-tenant cluster can connect to localhost network ports, every API call requires a per-daemon secret token stored in a private directory (`mode 0700`), and all requests with non-local HTTP `Host` headers are rejected to prevent DNS rebinding attacks.

Table 5 (Appendix A) shows results from our live 16-scenario test suite (16/16 = 100% expected verdicts):

- **ClickFix Interception:** Pasted terminal attack commands are intercepted and halted prior to shell execution (`executed=False`).
- **Automated Quarantine:** Malicious PHP web shells dropped into staging directories are quarantined into read-only files (`mode 0400`) within < 500 ms.

Table 2: Pre-Registered Rounds `v1.4→v2.0`. Each round replays one fresh seed's variants through the version before and after the change. "Contained" = block + hold that paused, quarantined or killed something; Wilson 95% intervals; neutral benign variants carry no mutation the specification forbids.

Round	Seed	Structural change (design decision; motivating finding)	Contained, before → after	Neutral false blocks	Benign changed
`v1.4→v1.5`	`833754500`	lineage-aware destination trust, incidents end, bounded state (A60–A62; F2)	630 → 631/637, 99.1% [98.0, 99.6]	1/158 (0.6%)	none
`v1.5→v1.6`	`545728926`	labeled block with no labeled process acts on its producers (A63; F11)	623 → 624/636, 98.1% [96.7, 98.9]	1/161 (0.6%)	none
`v1.6→v1.7`	`616687759`	every live label handed over (A64; property test)	623 → 623/634, 98.3% [96.9, 99.0]	3/154 (1.9%)	none
`v1.7→v2.0`	`171139148`	trust as lineage, channel roles relay (A65, A66; F10)	625 → 625/635, 98.4% [97.1, 99.1]	**0/158** (**0.0**% [0.0, 2.4])	5, all improved

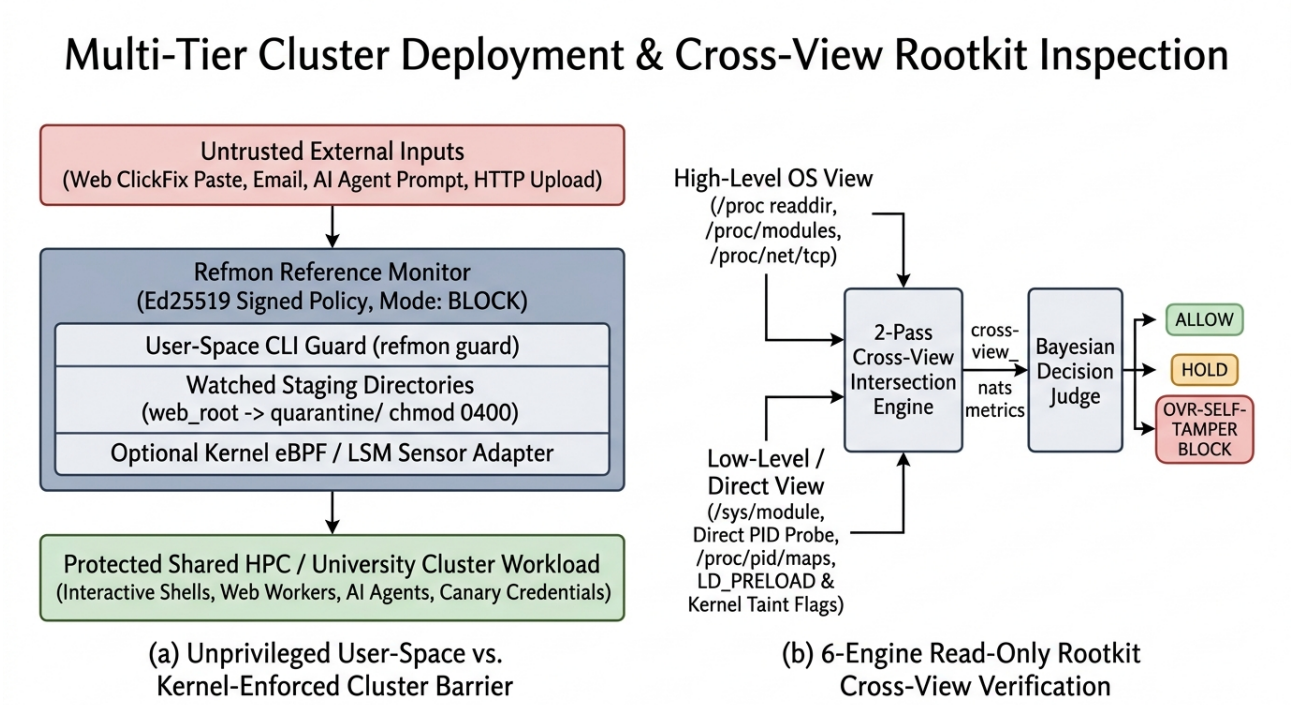


Figure 4: Multi-tier deployment architecture and host integrity verification: (a) unprivileged user-space command barrier (`refmon guard`) and watched staging directories (`web_root` → `quarantine/` mode `0400`) alongside optional kernel eBPF/LSM sensors; (b) six-engine two-pass cross-view rootkit sensor feeding bounded log-odds into the Bayesian decision judge.

- **Ransomware Suppression:** Sudden bursts of file modifications trigger automatic `mass_change` blocks.
- **Benign Workload Freedom:** Legitimate operations (such as installing Python packages via PyPI or running local benchmarks) proceed with zero false alarms.

In host integrity scans, the rootkit engine suppresses benign GPU driver kernel taints ($L = -6.91 \mapsto$ `ALLOW`), places unusual anonymous memory regions on hold ($L = -4.71 \mapsto$ `HOLD`), and blocks preload implants, hidden kernel modules, and stealth processes ($L \geq -2.51 \mapsto$ `BLOCK`).

Pre-Execution Command Intent Prediction. To protect interactive terminals, Refmon inspects commands *before* the shell executes them. The barrier analyzes typed commands to predict the events they would produce, checking for hidden network tunnels (`socat`, netcat, reverse SSH shells), attempts to read sensitive credentials (`.aws`, `.kube`, private SSH keys), modifications to startup scripts or git hooks, and attempts to wipe shell history.

To counter common shell obfuscation techniques, Refmon evaluates commands both as written and after stripping quotes, backslashes, and variable insertions (such as `${IFS}`). Table 4 (Appendix A) summarizes this coverage across 69 diverse probe commands.

4.5 Adversarial Privacy Audit and Test Suite Summary

Running `refmon audit` executes an active red-team privacy audit across all generated verdicts, reports, stories, buffered events, learning proposals, and golden files:

- **Cleartext Leakage:** 264 output artefacts scanned $\rightarrow$ 0 **raw content findings**.
- **Dictionary HMAC Attack:** 349 fingerprints attacked across $2,119$ common command/URL guesses without $K_{\text{dev}} \rightarrow 0$ **reversals** (whereas a positive control given K_{dev} reverses $6/6$, confirming attack-harness validity).
- **Four-Eyes Escrow:** 10 single-approver re-identification attempts $\rightarrow 0$ **permitted**; dual-approver requests succeed with hash-chain integrity verified (`audit chain intact = True`).
- **Automated Verification:** Across unit, Hypothesis property-based, golden replay, variant-matrix, rootkit, command-intent, CLI, and live HTTP tests, over $1,860$ automated tests pass.

5 Discussion

Attacks change payloads constantly, but what they do after gaining a foothold stays tied to OS rules: a web worker spawning a shell or dropping scripts will always break its role contract, no matter how clever the exploit. Refmon catches these actions by checking what happened instead of guessing what the payload looked like, while leaving ordinary developer commands alone. Fixing evidence monotonicity in `v1.3` and scoping trust to process lineage in `v2.0` closed edge-case leaks and cut false blocks to zero without widening permissions. Refmon keeps decisions deterministic rather than using reinforcement learning at runtime, avoiding reward poisoning and unexplainable black-box actions. Where gaps remain—such as out-of-band typed commands or slow leaks spread across hours—Refmon holds for human review rather than guessing, providing a reliable defense on shared nodes until kernel eBPF hooks can be deployed.

6 Future Work

We plan to hook Refmon directly into Linux `eBPF` LSM and `fanotify` for non-bypassable kernel-level mediation across all processes. We are also extending provenance tracking across multi-node Slurm and MPI batch jobs on shared clusters, tuning multi-day drift detectors for ultra-slow ransomware, and backing policy keys with TPM hardware roots of trust before running a longitudinal campus pilot.

7 Conclusion

Refmon demonstrates that tracking behavioral consequences rather than exploit payloads provides a reliable, privacy-preserving defense against zero-day exploits, ClickFix lures, and stealth rootkits. Evaluated across 46 golden scenarios, 886 mutation variants, and a live cluster barrier suite, Refmon achieves 100% golden accuracy and 98.4% attack containment with zero false blocks on benign workloads. Its mathematically verified monotonicity, minimal Bayes risk, and in-memory privacy boundary make it a practical, robust security foundation for shared computing clusters.

8 AI Disclosure

Generative AI assistants (Claude Code and Antigravity) were used under human supervision to help write code, generate test suites, render diagrams, and typeset this document from a human-written specification. No AI or machine-learning model runs inside the Refmon decision engine; every runtime verdict is a deterministic calculation governed by signed policies and proven mathematics. All signal weights were frozen prior to evaluation, mutation streams used pre-registered random seeds, policy keys remained strictly human-held, and all empirical claims were validated by executable test suites.

References

[1] J. P. Anderson, "Computer security technology planning study," Electronic Systems Division, Air Force Systems Command, Tech. Rep. ESD-TR-73-51, Vol. II, Oct. 1972.

[2] D. E. Denning, "A lattice model of secure information flow," *Communications of the ACM*, vol. 19, no. 5, pp. 236–243, May 1976.

[3] A. Wald, "Sequential tests of statistical hypotheses," *The Annals of Mathematical Statistics*, vol. 16, no. 2, pp. 117–186, Jun. 1945.

[4] E. S. Page, "Continuous inspection schemes," *Biometrika*, vol. 41, no. 1/2, pp. 100–115, Jun. 1954.

[5] G. Lorden, "Procedures for reacting to a change in distribution," *The Annals of Mathematical Statistics*, vol. 42, no. 6, pp. 1897–1908, Dec. 1971.

[6] I. J. Good, "The population frequencies of species and the estimation of population parameters," *Biometrika*, vol. 40, no. 3/4, pp. 237–264, Dec. 1953.

[7] J. Pearl, *Probabilistic Reasoning in Intelligent Systems: Networks of Plausible Inference*. San Mateo, CA: Morgan Kaufmann, 1988.

[8] B. H. Bloom, "Space/time trade-offs in hash coding with allowable errors," *Communications of the ACM*, vol. 13, no. 7, pp. 422–426, Jul. 1970.

[9] M. Bellare, R. Canetti, and H. Krawczyk, "Keying hash functions for message authentication," in *Advances in Cryptology — CRYPTO '96*, LNCS vol. 1109, pp. 1–15, Springer, 1996.

[10] D. J. Bernstein, N. Duif, T. Lange, P. Schwabe, and B.-Y. Yang, "High-speed high-security signatures," *Journal of Cryptographic Engineering*, vol. 2, no. 2, pp. 77–89, Sep. 2012.

[11] C. Dwork, F. McSherry, K. Nissim, and A. Smith, "Calibrating noise to sensitivity in private data analysis," in *Proc. Theory of Cryptography Conf. (TCC)*, LNCS vol. 3876, pp. 265–284, Springer, 2006.

[12] L. Sweeney, "k-anonymity: A model for protecting privacy," *Int. J. Uncertainty, Fuzziness and Knowledge-Based Systems*, vol. 10, no. 5, pp. 557–570, Oct. 2002.

[13] J. Newsome and D. X. Song, "Dynamic taint analysis for automatic detection, analysis, and signature generation of exploits on commodity software," in *Proc. NDSS*, Feb. 2005.

[14] T. Pasquier, X. Han, M. Goldstein, T. Moyer, D. Eyers, M. Seltzer, and J. Bacon, "Practical whole-system provenance capture," in *Proc. ACM SoCC*, pp. 405–418, Sep. 2017.

[15] M. N. Hossain, S. M. Milajerdi, J. Wang, B. Eshete, R. Gjomemo, R. Sekar, S. Stolper, and V. N. Venkatakrishnan, "SLEUTH: Real-time attack scenario reconstruction from COTS audit data," in *Proc. USENIX Security Symp.*, pp. 487–504, Aug. 2017.

[16] S. M. Milajerdi, R. Gjomemo, B. Eshete, R. Sekar, and V. N. Venkatakrishnan, "HOLMES: Real-time APT detection through correlation of suspicious information flows," in *Proc. IEEE Symp. Security and Privacy (S&P)*, pp. 1137–1152, May 2019.

[17] X. Han, T. Pasquier, A. Bates, J. Mickens, and M. Seltzer, "UNICORN: Runtime provenance-based detector for advanced persistent threats," in *Proc. NDSS*, Feb. 2020.

[18] S. Smalley, C. Vance, and W. Salamon, "Implementing SELinux as a Linux Security Module," NAI Labs Report #01-043, Dec. 2001.

[19] M. Bauer, "Paranoid penguin: An introduction to Novell AppArmor," *Linux Journal*, vol. 2006, no. 148, p. 13, Aug. 2006.

[20] J. Corbet, "Seccomp filter (Mode 2 BPF syscall filtering)," Linux Kernel Documentation (`Documentation/userspace-api/seccomp_filter.rst`), 2012.

[21] Y.-M. Wang, D. Beck, B. Vo, R. Roussev, and C. Verbowski, "Detecting stealth software with Strider GhostBuster," in *Proc. IEEE/IFIP Int. Conf. Dependable Systems and Networks (DSN)*, pp. 368–377, Jun. 2005.

[22] T. Garfinkel and M. Rosenblum, "A virtual machine introspection based architecture for intrusion detection," in *Proc. NDSS*, pp. 191–206, Feb. 2003.

[23] J. Jung, V. Paxson, A. W. Berger, and H. Balakrishnan, "Fast portscan detection using sequential hypothesis testing," in *Proc. IEEE Symp. Security and Privacy (S&P)*, pp. 211–225, May 2004.

[24] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection," in *Proc. ACM AISec Workshop*, pp. 79–90, Nov. 2023.

[25] MITRE Corporation, "MITRE ATT&CK Enterprise Matrix (T1204.004 User Execution: Malicious Copy and Paste)," Technical Knowledge Base, 2024–2026.

[26] E. B. Wilson, "Probable inference, the law of succession, and statistical inference," *Journal of the American Statistical Association*, vol. 22, no. 158, pp. 209–212, Jun. 1927.

[27] Q. McNemar, "Note on the sampling error of the difference between correlated proportions or percentages," *Psychometrika*, vol. 12, no. 2, pp. 153–157, Jun. 1947.

A Supplementary Architecture, Implementation, and Verification Data

This appendix provides the end-to-end pipeline architecture figure, the frozen evidence calculus weights, the user-space command barrier loop, and the complete verification tables referenced in the main text.

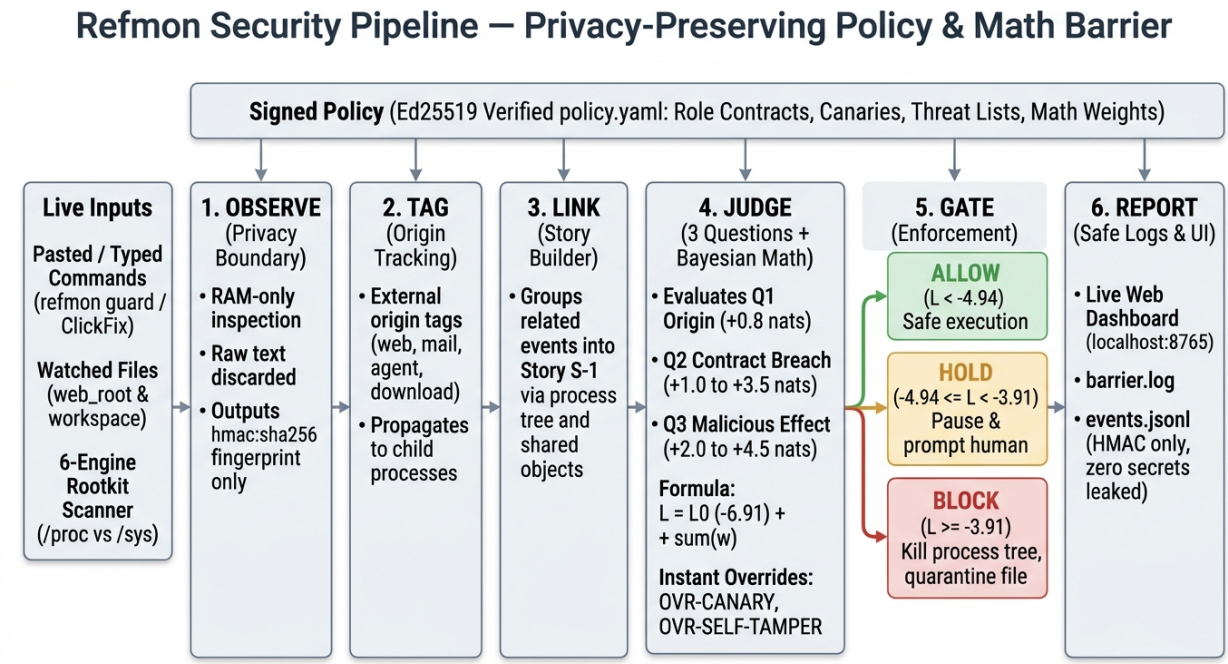


Figure 5: Architectural overview of the six-stage Refmon Security Pipeline. Raw inputs are processed strictly in volatile memory during Stage 1 (`Observe`) and reduced to keyed `hmac:sha256` digests before reaching provenance tagging, story linking, Bayesian judging, containment gating, and privacy reporting.

Table 3: Frozen `v1.3` Policy Signal Weights (w_j in nats) and Cost Parameters ($\pi = 0.001$, $C_{\text{miss}} = 1000$, $C_{\text{fb}} = 20$, $C_{\text{hold}} = 5$, $h = 0.7$).

Axis	Signal Name	w_j	Signal Name	w_j
Origin	`origin_external`	0.8	—	—
Contract	`external_exec`	1.0	`write_other`	1.0
	`unsigned_launch`	1.5	`surprisal`	2.0
	`universal_violation`	2.0	`exec_violation`	2.5
	`connect_violation`	2.5	`asset_violation`	2.5
	`web_worker_exec`	3.0	`write_executable`	3.5
	`persist_violation`	3.5	`privilege_violation`	3.5
Effect	`asset_access`	1.5	`response_anomaly`	1.5
	`hide`	2.0	`fetch`	2.5
	`persist`	2.5	`exfil`	2.5
	`escalate`	2.5	`mass_change`	2.5
	`tamper`	3.0	`lateral`	3.5
	`credential`	3.5	`resp_anom_repeated`	3.5
	`threat_list_hit`	4.5	—	—

Listing 1: Core unprivileged user-space command barrier loop (excerpted from `src/refmon/live.py`).

```
story_id, j = self._feed_raw_events(raw_events)  # Observe -> Tag -> Link -> Judge -> Gate
lo = float(j.log_odds if j.log_odds is not None else 0.0)
blocked_by_barrier = (
    self.policy.mode is Mode.BLOCK
    and j.decision in (Decision.BLOCK, Decision.HOLD)
)
```

```
if execute and not blocked_by_barrier and j.decision is Decision.ALLOW:
    proc = subprocess.run(cmd, shell=True, cwd=str(self.workspace_dir),
                          capture_output=True, text=True, timeout=5.0, check=False)
```

Table 4: Command-Barrier Probes Typed Locally (`mode=block`). Bold: the held-out measurement.

Probe set	**v1.7**	**v2.0 first run**	**v2.0 now**
23 attacker behaviours (`test_intent.py`)	23/23	23/23	23/23
20 evasions, written before A67	11/20	20/20	20/20
20 attacks, held out	13/20	**15/20**	20/20
23 + 14 benign look-alikes (false alarms)	1	0	0
15 benign, held out (false alarms)	1	**1**	0

Table 5: Live Barrier 16-Scenario Verification Suite (`mode=block`).

Scenario Type	**Origin / Role**	**Q-Fails**	L **(nats)**	**Verdict**
ClickFix Paste (`curl|sh`)	`web` / `shell`	3/3	−0.61	`BLOCK`
Obfuscated Base64 Mail	`mail` / `shell`	3/3	−3.11	`BLOCK`
AI Prompt-Injection Abuse	`agent_in` / `agent`	3/3	−0.11	`BLOCK`
Protected Credential Theft	`mail` / `shell`	OVR	−1.61	`BLOCK`
Server Web Worker RCE	`req_in` / `web`	3/3	−0.61	`BLOCK`
Web Shell Drop (`.php`)	`req_in` / `web`	3/3	−0.11	`BLOCK+QUAR`
Ransomware 55-File Burst	local / `shell`	2/3	−2.41	`BLOCK`
Decoy Canary Credential	`web` / `shell`	OVR	−1.61	`BLOCK`
SUID / `sudo` Escalation	`dl` / `shell`	3/3	−2.61	`BLOCK`
Crontab / `.bashrc` Persist	`web` / `shell`	3/3	−2.61	`BLOCK`
Security Tamper (`pkill`)	local / `shell`	OVR	−1.91	`BLOCK`
Supply-Chain Post-Install	`pkg` / `installer`	3/3	+1.89	`BLOCK`
Unverified Mirror Fetch	local / `shell`	1/3	−4.41	`HOLD`
Benign Local `ls -la`	local / `shell`	0/3	−6.91	`ALLOW`
Benign Pasted `echo`	`web` / `shell`	2/3	−5.11	`ALLOW`
Trusted PyPI Download	`dl` / `installer`	OVR	−6.91	`ALLOW`

B Proofs of Provenance Decay and Noisy-OR Fusion

Proposition B.1 (Monotonicity and Invariance of Label Decay)**.** *Let* $s_\ell(t) = s_0 \cdot 2^{-(t-t_0)/T_{1/2}}$ *for* $s_0 \in [0,1]$, $T_{1/2} > 0$, *and* $t \geq t_0$. *Then:*

1. $s_\ell(t) \in [0, s_0] \subseteq [0,1]$ *for all* $t \geq t_0$*;*
2. $s_\ell(t)$ *is strictly decreasing in* t *whenever* $s_0 > 0$*, with* $s_\ell(t_0 + kT_{1/2}) = 2^{-k} s_0$*;*
3. *Under edge attenuation* $a_e \in (0,1]$*, any* d*-hop causal path* $p = (e_1, \ldots, e_d)$ *satisfies* $s_{leaf}(t) = s_0\big(\prod_{r=1}^{d} a_{e_r}\big) 2^{-\Delta t/T_{1/2}} \leq s_\ell(t)$.

Proof. Since $T_{1/2} > 0$ and $\Delta t = t - t_0 \geq 0$, we have $-\Delta t/T_{1/2} \leq 0$, so $0 < 2^{-\Delta t/T_{1/2}} \leq 1$, proving $0 \leq s_\ell(t) \leq s_0 \leq 1$. Differentiating with respect to t yields

$$\frac{\mathrm{d}s_\ell(t)}{\mathrm{d}t} = -\frac{\ln 2}{T_{1/2}}\, s_0\, 2^{-(t-t_0)/T_{1/2}} < 0 \quad (\forall s_0 > 0),$$

establishing strict temporal monotonicity, and direct substitution of $\Delta t = kT_{1/2}$ gives $s_\ell(t_0 + kT_{1/2}) = 2^{-k} s_0$. Finally, because $a_{e_r} \in (0,1]$ for every edge e_r, induction over path length d yields $0 < \prod_{r=1}^{d} a_{e_r} \leq 1$, whence $s_{\text{leaf}}(t) \leq s_\ell(t)$. □

Theorem B.2 (Algebraic and Dominance Properties of Noisy-OR)**.** *For any finite multiset of label strengths* $\mathbf{s} = (s_1, \ldots, s_m) \in [0,1]^m$*, define* $\mathrm{NOR}(\mathbf{s}) = 1 - \prod_{i=1}^{m}(1 - s_i)$ *(with empty product* 1*). Then:*

1. ***Range closure:*** $\mathrm{NOR}(\mathbf{s}) \in [0,1]$*;*
2. ***Dominance:*** $\max_{1\le i\le m} s_i \le \mathrm{NOR}(\mathbf{s}) \le \min\big(1, \sum_{i=1}^m s_i\big)$*;*
3. ***Commutative monoid:*** NOR *is symmetric, associative under binary composition* $x \oplus y = 1-(1-x)(1-y) = x+y-xy$*, has identity element* 0*, and absorbing element* 1*;*
4. ***Coordinate-wise monotonicity:*** *If* $s_i \le s_i'$ *for all* i*, then* $\mathrm{NOR}(\mathbf{s}) \le \mathrm{NOR}(\mathbf{s}')$*.*

Proof. (1) Since $s_i \in [0,1]$, each factor satisfies $1-s_i \in [0,1]$, so $P = \prod_{i=1}^m (1-s_i) \in [0,1]$ and $1-P \in [0,1]$.

(2) Fix an arbitrary index $k \in \{1,\dots,m\}$. Because $\prod_{i\neq k}(1-s_i) \le 1$ and $1-s_k \ge 0$, we obtain

$$\prod_{i=1}^m (1-s_i) = (1-s_k)\prod_{i\neq k}(1-s_i) \le 1-s_k$$
$$\Longrightarrow\ \mathrm{NOR}(\mathbf{s}) \ge s_k.$$

Taking the maximum over k yields $\mathrm{NOR}(\mathbf{s}) \ge \max_i s_i$. For the upper bound, induction on m using Weierstrass's inequality $\prod_{i=1}^m (1-s_i) \ge 1 - \sum_{i=1}^m s_i$ gives $\mathrm{NOR}(\mathbf{s}) \le \sum_{i=1}^m s_i$, and combining with (1) gives $\min(1, \sum_i s_i)$.

(3) Symmetry follows from commutativity of real multiplication. For associativity, $(x\oplus y)\oplus z = 1-(1-[1-(1-x)(1-y)])(1-z) = 1-(1-x)(1-y)(1-z) = x\oplus(y\oplus z)$. Moreover, $x\oplus 0 = 1-(1-x)\cdot 1 = x$ and $x\oplus 1 = 1-(1-x)\cdot 0 = 1$.

(4) If $s_i \le s_i'$, then $0 \le 1-s_i' \le 1-s_i$ for each i, so $\prod_{i=1}^m (1-s_i') \le \prod_{i=1}^m (1-s_i)$, whence $\mathrm{NOR}(\mathbf{s}) \le \mathrm{NOR}(\mathbf{s}')$. □

C Derivation of Bayes-Optimal Thresholds and Contract Gate

Theorem C.1 (Bayes Minimal-Risk Log-Odds Thresholds)**.** *Let* $Y \in \{0,1\}$ *denote whether a story* S *is benign (*$Y=0$*) or an attack (*$Y=1$*), and let* $p = \mathbb{P}(Y=1 \mid S) = (1+e^{-L})^{-1}$ *be the posterior attack probability corresponding to log-odds* $L = \ln \frac{p}{1-p}$*. Suppose:*

- *Action* a_{allow} *incurs loss* $C_{miss} > 0$ *if* $Y=1$ *and* 0 *if* $Y=0$*;*
- *Action* a_{block} *incurs loss* 0 *if* $Y=1$ *and* $C_{fb} > 0$ *if* $Y=0$*;*
- *Action* a_{hold} *incurs human-review cost* $C_{hold} > 0$ *if* $Y=0$*, and catches a true attack (*$Y=1$*) with probability* $h \in (0,1]$ *(leaving residual miss cost* $(1-h)C_{miss}$*; when* $C_{hold} \ll C_{miss}$ *we neglect the* $O(C_{hold})$ *operational overhead under* $Y=1$*).*

Then:

1. *Pairwise preference* $R(a_{block} \mid S) \le R(a_{allow} \mid S)$ *holds iff*
$$L \ge \tau_{block} \triangleq \ln \frac{C_{fb}}{C_{miss}};$$
2. *Pairwise preference* $R(a_{hold} \mid S) \le R(a_{allow} \mid S)$ *holds iff*
$$L \ge \tau_{hold} \triangleq \ln \frac{C_{hold}}{h \cdot C_{miss}};$$
3. *A non-empty* `HOLD` *interval* $[\tau_{hold}, \tau_{block})$ *exists iff* $C_{hold} < h \cdot C_{fb}$*.*

Proof. The conditional Bayes risks of the three actions given posterior $p \in (0,1)$ are:

$$R(a_{\text{allow}} \mid S) = p\, C_{\text{miss}},$$
$$R(a_{\text{block}} \mid S) = (1-p)\, C_{\text{fb}},$$
$$R(a_{\text{hold}} \mid S) = p(1-h)C_{\text{miss}} + (1-p)C_{\text{hold}}.$$

For (1), $R(a_{\text{block}} \mid S) \le R(a_{\text{allow}} \mid S)$ iff $(1-p)C_{\text{fb}} \le p\, C_{\text{miss}}$, which rearranges to

$$\frac{p}{1-p} \ge \frac{C_{\text{fb}}}{C_{\text{miss}}} \iff L = \ln \frac{p}{1-p} \ge \ln \frac{C_{\text{fb}}}{C_{\text{miss}}} = \tau_{\text{block}}.$$

For (2), $R(a_{\text{hold}} \mid S) \le R(a_{\text{allow}} \mid S)$ iff

$$p(1-h)C_{\text{miss}} + (1-p)C_{\text{hold}} \le p\,C_{\text{miss}}$$
$$\iff (1-p)C_{\text{hold}} \le p\,h\,C_{\text{miss}}.$$

Dividing by $(1-p)hC_{\text{miss}} > 0$ and taking natural logarithms yields

$$\frac{p}{1-p} \ge \frac{C_{\text{hold}}}{h \cdot C_{\text{miss}}} \iff L \ge \ln \frac{C_{\text{hold}}}{h \cdot C_{\text{miss}}} = \tau_{\text{hold}}.$$

For (3), by strict monotonicity of $\ln(\cdot)$, $\tau_{\text{hold}} < \tau_{\text{block}} \iff \frac{C_{\text{hold}}}{hC_{\text{miss}}} < \frac{C_{\text{fb}}}{C_{\text{miss}}} \iff C_{\text{hold}} < h\,C_{\text{fb}}$. Substituting Refmon's default policy costs ($C_{\text{hold}} = 5$, $h = 0.7$, $C_{\text{fb}} = 20$) gives $5 < 14$, yielding $\tau_{\text{hold}} = \ln(5/700) = -4.9416 < \tau_{\text{block}} = \ln(20/1000) = -3.9120$. □

Corollary C.2 (Sensitivity Directionality)**.** *$\partial\tau_{block}/\partial C_{fb} = 1/C_{fb} > 0$, $\partial\tau_{block}/\partial C_{miss} = -1/C_{miss} < 0$, and $\partial\tau_{hold}/\partial h = -1/h < 0$. Thus raising false-block penalties raises the evidence bar for blocking, whereas raising missed-attack cost lowers both thresholds monotonically.*

D Proof of Posterior Monotonicity in `v1.3` (Resolution of F8)

Theorem D.1 (Monotonicity of `v1.3` Calculus vs. Non-Monotonicity of `v1.2`)**.** *Fix a story S with non-empty Contract or Effect signals having positive max weights $W_C = \max_{j \in \mathcal{S}_C} w_j > 0$ and $W_E = \max_{k \in \mathcal{S}_E} w_k \ge 0$, origin weight $w_{ori} \ge 0$, and origin threshold $\theta \in (0,1)$.*

1. *Under the legacy `v1.2` rule $L_{v1.2}(s) = L_0 + w_{ori} s\mathbf{1}[s \ge \theta] + \big(s\mathbf{1}[s \ge \theta] + \mathbf{1}[s < \theta]\big)(W_C + W_E)$, $L_{v1.2}(s)$ is **not** non-decreasing on $[0,1]$ whenever $W_C + W_E > \frac{\theta}{1-\theta} w_{ori}$.*
2. *Under the `v1.3` rule $L_{v1.3}(s) = L_0 + w_{ori} s\mathbf{1}[s \ge \theta] + W_C + W_E$, $L_{v1.3}(s)$ is monotonically non-decreasing over all $s \in [0,1]$ and under arbitrary signal additions with $w_j \ge 0$.*

Proof. (1) Evaluate the left and right limits of $L_{\text{v1.2}}(s)$ at $s = \theta$:

$$\lim_{s \uparrow \theta} L_{\text{v1.2}}(s) = L_0 + W_C + W_E,$$
$$L_{\text{v1.2}}(\theta) = L_0 + \theta w_{\text{ori}} + \theta(W_C + W_E).$$

Hence $L_{\text{v1.2}}(\theta) - \lim_{s \uparrow \theta} L_{\text{v1.2}}(s) = \theta w_{\text{ori}} - (1-\theta)(W_C + W_E)$. For Refmon's defaults ($\theta = 0.5$, $w_{\text{ori}} = 0.8$), whenever $W_C + W_E > 0.8$ (which holds for *every* Contract + Effect pair in Table 3, e.g., $W_C + W_E = 1.0 + 2.5 = 3.5$), we have $L_{\text{v1.2}}(0.5) - L_{\text{v1.2}}(0.499) = 0.4 - 1.75 = -1.35 < 0$, proving a downward jump where stronger provenance produced weaker evidence.

(2) In `v1.3`, write $g(s) = w_{\text{ori}} s\mathbf{1}[s \ge \theta]$. For $0 \le s_1 \le s_2 < \theta$, $g(s_1) = g(s_2) = 0$. For $0 \le s_1 < \theta \le s_2 \le 1$, since $w_{\text{ori}} \ge 0$ and $s_2 \ge \theta > 0$, $g(s_1) = 0 \le w_{\text{ori}} s_2 = g(s_2)$. For $\theta \le s_1 \le s_2 \le 1$, $g(s_2) - g(s_1) = w_{\text{ori}}(s_2 - s_1) \ge 0$. Since W_C and W_E are independent of s (apart from D23's indicator $\mathbf{1}[s \ge \theta]$ which only *adds* positive allowed-effect weights when s crosses θ), $L_{\text{v1.3}}(s_1) \le L_{\text{v1.3}}(s_2)$ for all $0 \le s_1 \le s_2 \le 1$. Furthermore, adding any signal j^* with $w_{j^*} \ge 0$ to question set $\mathcal{S}_q$ satisfies $\max_{j \in \mathcal{S}_q \cup \{j^*\}} w_j s_j \ge \max_{j \in \mathcal{S}_q} w_j s_j$, so $L_{\text{v1.3}}$ never decreases when evidence is added. □

E Proofs of Laplace Surprisal and Good-Turing Unseen Mass

Proposition E.1 (Properness and Surprisal Bounds of Laplace Role Model)**.** *Let $n_1, \ldots, n_K \ge 1$ be observed counts summing to $N = \sum_{e=1}^{K} n_e$, and define $\hat{p}(e) = \frac{n_e + \alpha}{N + \alpha(K+1)}$ for $e \in \{1, \ldots, K\}$ and $\hat{p}(\textit{unseen}) = \frac{\alpha}{N + \alpha(K+1)}$ with $\alpha > 0$. Then $\hat{p}$ is a valid probability distribution on $\{1, \ldots, K, \textit{unseen}\}$, and unseen-event surprisal satisfies $I(\textit{unseen}) = \log_2\big(\frac{N}{\alpha} + K + 1\big)$.*

Proof. Every probability is strictly positive since $n_e \ge 0$ and $\alpha > 0$. Summing over all $K+1$ outcomes gives

$$\sum_{e=1}^{K} \frac{n_e + \alpha}{N + \alpha(K+1)} + \frac{\alpha}{N + \alpha(K+1)}$$
$$= \frac{\sum_{e=1}^{K} n_e + K\alpha + \alpha}{N + \alpha(K+1)} = \frac{N + \alpha(K+1)}{N + \alpha(K+1)} = 1.$$

Taking $-\log_2 \hat{p}(\text{unseen})$ yields $\log_2((N+\alpha(K+1))/\alpha) = \log_2(N/\alpha+K+1)$. In particular, for $\alpha = 1$, $I(\text{unseen}) \geq 12$ bits requires $N + K + 1 \geq 2^{12} = 4096$, automatically preventing surprisal false alarms on small samples even before the Good-Turing check. □

Theorem E.2 (Unbiasedness of Good-Turing Unseen Probability Estimator [6])**.** *Suppose N i.i.d. events are drawn from an unknown discrete distribution $\{p_e\}_{e\in\mathcal{U}}$ over a countable alphabet $\mathcal{U}$. Let N_e be the count of kind e in the first N draws, $U_N = \sum_{e\in\mathcal{U}} p_e \mathbf{1}[N_e = 0]$ be the true random probability mass of kinds unseen in N draws, and $N_1^{(N+1)} = \sum_{e\in\mathcal{U}} \mathbf{1}[N'_e = 1]$ be the number of singletons in a sample of size $N+1$. Then:*

$$\mathbb{E}[U_N] = \frac{\mathbb{E}\big[N_1^{(N+1)}\big]}{N+1} \approx \frac{\mathbb{E}[N_1^{(N)}]}{N}.$$

Proof. By linearity of expectation and $N_e \sim \text{Binomial}(N, p_e)$,

$$\mathbb{E}[U_N] = \sum_{e\in\mathcal{U}} p_e \mathbb{P}(N_e = 0) = \sum_{e\in\mathcal{U}} p_e (1-p_e)^N.$$

Conversely, in $N+1$ i.i.d. draws, $\mathbb{P}(N'_e = 1) = (N+1)p_e(1-p_e)^N$. Summing over $e \in \mathcal{U}$ and dividing by $N+1$ gives

$$\begin{aligned}\frac{\mathbb{E}\big[N_1^{(N+1)}\big]}{N+1} &= \frac{1}{N+1}\sum_{e\in\mathcal{U}} (N+1)p_e(1-p_e)^N \\ &= \sum_{e\in\mathcal{U}} p_e(1-p_e)^N = \mathbb{E}[U_N].\end{aligned}$$

For $N \gg 1$, replacing $N+1$ by N yields $\hat{p}_{\text{unseen}} = N_1/N$ with $O(1/N)$ bias. □

F Proofs of Statistical Rate and CUSUM Drift Properties

Proposition F.1 (Numerical Exactness of Upper-Tail Poisson Recurrence)**.** *For $X \sim \text{Poisson}(\lambda)$ with $\lambda > 0$ and integer $k \geq 1$, the upper tail $T(k,\lambda) = \sum_{i=k}^{\infty} t_i$ with $t_k = \exp(-\lambda + k\ln\lambda - \ln\Gamma(k+1))$ and recurrence $t_{i+1} = t_i \cdot \frac{\lambda}{i+1}$ converges strictly monotonically for $i \geq \lceil\lambda\rceil$ with relative truncation error below machine epsilon ε_{mach} once $t_i \leq \varepsilon_{mach} \sum_{j=k}^{i} t_j$.*

Proof. Computing $\ln t_k$ via $\text{lgamma}(k+1) = \ln(k!)$ avoids catastrophic cancellation in $1 - \sum_{i=0}^{k-1} t_i$ when $T(k,\lambda) \approx 10^{-12} \ll \varepsilon_{\text{mach}} \approx 2.22 \times 10^{-16}$ (where complement subtraction returns 0.0). For all $i \geq \max(k, \lceil 2\lambda\rceil)$, $t_{i+1}/t_i = \lambda/(i+1) \leq 1/2$, so the omitted tail satisfies $\sum_{j=i+1}^{\infty} t_j \leq t_{i+1}\sum_{r=0}^{\infty} 2^{-r} = 2t_{i+1} \leq t_i$, bounding relative error by $\varepsilon_{\text{mach}}$. □

Proposition F.2 (Negative Drift Under Benign Null and Poisoning Immunity)**.** *Let $z_t = (x_t - \mu)/\sigma$ have $\mathbb{E}_0[z_t] = 0$ under benign traffic and $\mathbb{E}_1[z_t] = \delta > k_c > 0$ under slow exfiltration. Then the one-sided CUSUM $S_t = \max(0, S_{t-1} + z_t - k_c)$ has strictly negative pre-reflection increment drift $\mathbb{E}_0[z_t - k_c] = -k_c < 0$ under $\mathcal{H}_0$ (preventing unbounded random-walk drift) and strictly positive drift $\mathbb{E}_1[z_t - k_c] = \delta - k_c > 0$ under $\mathcal{H}_1$, matching the Gaussian log-likelihood ratio test for shift $\delta = 2k_c$ [4, 5]. Moreover, when EMA updates $\hat{\lambda}_t = (1-\beta)\hat{\lambda}_{t-1} + \beta y_t \cdot \mathbf{1}[\neg\textit{incident}]$ are gated on J.decision = `ALLOW`, any sequence of anomalous bursts during an open incident leaves $\hat{\lambda}_t \equiv \hat{\lambda}_{t_{open}}$, preventing baseline inflation.*

Proof. Under Gaussian $z_t \sim \mathcal{N}(\delta, 1)$ vs. $\mathcal{N}(0,1)$, the single-step log-likelihood ratio is $\ln\frac{\phi(z_t-\delta)}{\phi(z_t)} = \delta z_t - \frac{\delta^2}{2} = \delta\big(z_t - \frac{\delta}{2}\big)$. Setting $k_c = \delta/2 > 0$ scales Page's SPRT recursion directly into Eq. (17). Under $\mathcal{H}_0$, $\mathbb{E}_0[z_t - k_c] = -k_c < 0$; under $\mathcal{H}_1$ with $\delta > k_c$, $\mathbb{E}_1[z_t - k_c] = \delta - k_c > 0$. Finally, during $[t_{\text{open}}, t_{\text{close}}]$, $\mathbf{1}[\neg\text{incident}] = 0 \implies \hat{\lambda}_t = \hat{\lambda}_{t_{\text{open}}}$ by trivial induction. □

G Proofs of Differential Privacy and Zero-FP Bloom Filtering

Theorem G.1 (ε-Differential Privacy of Aggregate Fleet Counts)**.** *Let D, D' be neighboring device-telemetry datasets differing in the state of at most one device, and let $f(D) \in \mathbb{N}$ count the number of devices experiencing a given rule or tactic ($\Delta f = \max_{D\sim D'} |f(D) - f(D')| = 1$). Then the mechanism $\mathcal{M}(D) = f(D) + \eta$ where η is sampled via inverse transform $F^{-1}(u) = -\frac{1}{\varepsilon}\operatorname{sgn}(u - \frac{1}{2})\ln\big(1 - 2|u - \frac{1}{2}|\big)$ for $u \sim \text{Uniform}(0,1)$ satisfies pure ε-differential privacy.*

Proof. First, for $y \geq 0$, $\mathbb{P}(F^{-1}(u) \geq y) = \mathbb{P}\big(u - \frac{1}{2} \geq \frac{1}{2}(1 - e^{-\varepsilon y})\big) = \frac{1}{2}e^{-\varepsilon y}$; by symmetry around 0, η has density $p_\eta(z) = \frac{\varepsilon}{2}\exp(-\varepsilon|z|)$, i.e., $\eta \sim \mathrm{Lap}(0, 1/\varepsilon)$. Second, for any measurable output $z \in \mathbb{R}$ and neighboring D, D' with $|f(D) - f(D')| \leq \Delta f = 1$, the reverse triangle inequality $|z - f(D')| - |z - f(D)| \leq |f(D) - f(D')| \leq 1$ gives:

$$\begin{aligned}\frac{p_{\mathcal{M}(D)}(z)}{p_{\mathcal{M}(D')}(z)} &= \frac{\frac{\varepsilon}{2}\exp(-\varepsilon|z - f(D)|)}{\frac{\varepsilon}{2}\exp(-\varepsilon|z - f(D')|)} \\ &= \exp\Big(\varepsilon\big(|z - f(D')| - |z - f(D)|\big)\Big) \leq e^{\varepsilon}.\end{aligned}$$

□

Theorem G.2 (Optimal Sizing and Zero Operational False Positives of Confirmed Bloom Threat Lists)**.** *For n threat entries inserted into an m-bit array using k_h independent hash positions, the asymptotic filter false-positive probability $p(k_h) = \big(1 - e^{-k_h n/m}\big)^{k_h}$ is minimized at $k_h^* = \frac{m}{n}\ln 2$, requiring $m = \lceil -n \ln p_{fp}/(\ln 2)^2 \rceil$ bits for target hint rate p_{fp}. Moreover, two-stage confirmation $H(x) = \mathbf{1}[\mathcal{B}(x) = 1 \wedge d(x) \in \mathcal{D}_{exact}]$ achieves zero false negatives relative to $\mathcal{D}_{exact}$ and false-positive probability $\leq 2^{-256}$ (HMAC-SHA256 collision bound).*

Proof. Let $g(k_h) = \ln p(k_h) = k_h \ln(1 - e^{-k_h n/m})$. Setting $x = e^{-k_h n/m} \in (0, 1)$ gives $k_h = -\frac{m}{n}\ln x$ and $g = -\frac{m}{n}\ln x \ln(1 - x)$. Differentiating with respect to x:

$$\begin{aligned}\frac{\mathrm{d}g}{\mathrm{d}x} = -\frac{m}{n}\left(\frac{\ln(1-x)}{x} - \frac{\ln x}{1-x}\right) = 0 \\ \iff x = 1 - x \iff x = \frac{1}{2}.\end{aligned}$$

Substituting $x = 1/2$ yields $k_h^* = \frac{m}{n}\ln 2$ and $p(k_h^*) = (1/2)^{(m/n)\ln 2} = e^{-m(\ln 2)^2/n} \leq p_{\mathrm{fp}} \iff m \geq -n \ln p_{\mathrm{fp}}/(\ln 2)^2$. Finally, because a Bloom filter has zero false negatives ($d(x) \in \mathcal{D}_{\mathrm{exact}} \implies \mathcal{B}(x) = 1$), $H(x) = 1 \iff d(x) \in \mathcal{D}_{\mathrm{exact}}$; for $x \notin \mathcal{D}_{\mathrm{bad}}$, $d(x) \in \mathcal{D}_{\mathrm{exact}}$ occurs only on an HMAC-SHA256 collision across n entries ($\leq n \cdot 2^{-256}$ under PRF security [9]). □

H Bounds of the Trust Mark (`v2.0`, A65)

Proposition H.1 (Trust Never Outlives or Outreaches a Label)**.** *Let process p ingest trusted-root content at t_p, let $s_{floor} \in (0, 1)$ be the label alive floor, and let $\mathrm{trust}_p(t)$ be defined by Eq. (8). Then:*

1. ***Lifetime:*** *$\mathrm{trust}_p(t) = 1$ implies $t - t_p \leq T_{1/2}\log_2(1/s_{floor})$, the lifetime of a non-persistent label without contract evidence created at t_p;*
2. ***Cancellation:*** *if any label in $\mathrm{ever}(p)$ is alive at t, then $\mathrm{trust}_p(t) = 0$, whatever its strength relative to θ_{ori};*
3. ***Reach:*** *the mark reaches only p and the processes spawned in its lineage; a channel role that ingests trusted content does not acquire it, and no file or persistence item carries it, so a startup entry written under trust starts a later story without trust;*
4. ***Precedence:*** *no story containing a self-tamper, canary or protected-asset flag is allowed by the mark.*

Proof. (1) The first factor of Eq. (8) is 1 iff $2^{-(t-t_p)/T_{1/2}} \geq s_{\mathrm{floor}} \iff t - t_p \leq T_{1/2}\log_2(1/s_{\mathrm{floor}})$, which by Eq. (6) with $s_0 = 1$ is exactly the time at which such a label falls below the floor (Proposition B.1). With $T_{1/2} = 30$ min and $s_{\mathrm{floor}} = 0.02$ this is $30\log_2 50 \approx 169$ min. (2) The second factor is 0 by definition. (3) The mark is set on an ingesting process only when its role is not a channel role, and copied only when a new process is created with p (or a descendant) as parent; rule 5 (files and persistence items) copies labels, never the mark. (4) `Judge` evaluates the overrides in the fixed order self-tamper $\succ$ canary $\succ$ protected asset $\succ$ trusted root and returns the first that applies, so the trusted-root override is reached only when none of the blocking overrides is present. □

I Proof of Multi-Pass Rootkit Churn Suppression

Proposition I.1 (Exponential Suppression of Transient OS Race Discrepancies)**.** *Suppose during pass $r \in \{1, \ldots, m\}$ separated by $\Delta\tau$ exceeding typical syscall directory-walk latency, a benign short-lived entity x (such as an exiting worker PID or unloading kernel module) produces a transient cross-view race $x \in \mathcal{V}_{low}^{(r)} \setminus \mathcal{V}_{high}^{(r)}$ with probability $p_{race} \ll 1$ on*

Pass 1 and conditional persistence probability $q_{persist} \ll 1$ on each subsequent pass, whereas a stealth rootkit actively hiding x^* satisfies $x^* \in \mathcal{V}_{low}^{(r)} \setminus \mathcal{V}_{high}^{(r)}$ with probability $p_{tp} \approx 1$ on every pass. Then for $\Delta_{pers} = \bigcap_{r=1}^{m} (\mathcal{V}_{low}^{(r)} \setminus \mathcal{V}_{high}^{(r)})$:

$$\mathbb{P}(x \in \Delta_{pers} \mid \textit{benign}) = p_{race} \cdot q_{persist}^{m-1},$$
$$\ln \frac{\mathbb{P}(x \in \Delta_{pers} \mid \textit{rootkit})}{\mathbb{P}(x \in \Delta_{pers} \mid \textit{benign})} = \ln \frac{p_{tp}}{p_{race}} + (m-1) \ln \frac{1}{q_{persist}}.$$

Proof. By the chain rule of conditional probability under $\mathcal{H}_0$ (benign OS churn) across passes $1, \ldots, m$,

$$\mathbb{P}\left(\bigcap_{r=1}^{m} E_r \;\middle|\; \mathcal{H}_0 \right) = \mathbb{P}(E_1 \mid \mathcal{H}_0) \prod_{r=2}^{m} \mathbb{P}(E_r \mid E_{1..r-1}, \mathcal{H}_0)$$
$$= p_{\text{race}}\, q_{\text{persist}}^{m-1}.$$

Under a persistent rootkit hook (e.g., an LKM unlinked from `modules` list but still mounted in `/sys/module`, or `getdents64` filtering PID x^*), E_r holds deterministically across passes ($\mathbb{P}(\bigcap_{r=1}^{m} E_r \mid \text{rootkit}) = p_{\text{tp}}$). Taking the natural log-likelihood ratio yields $\ln(p_{\text{tp}}/p_{\text{race}}) + (m-1)\ln(1/q_{\text{persist}})$, and applying the policy cap $w_{\max}$ yields Eq. (19). For $p_{\text{race}} = 0.02$, $q_{\text{persist}} = 0.05$, and $m = 2$, benign race probability drops from 2×10^{-2} to 10^{-3} (20× reduction) in a single 50 ms interval. □